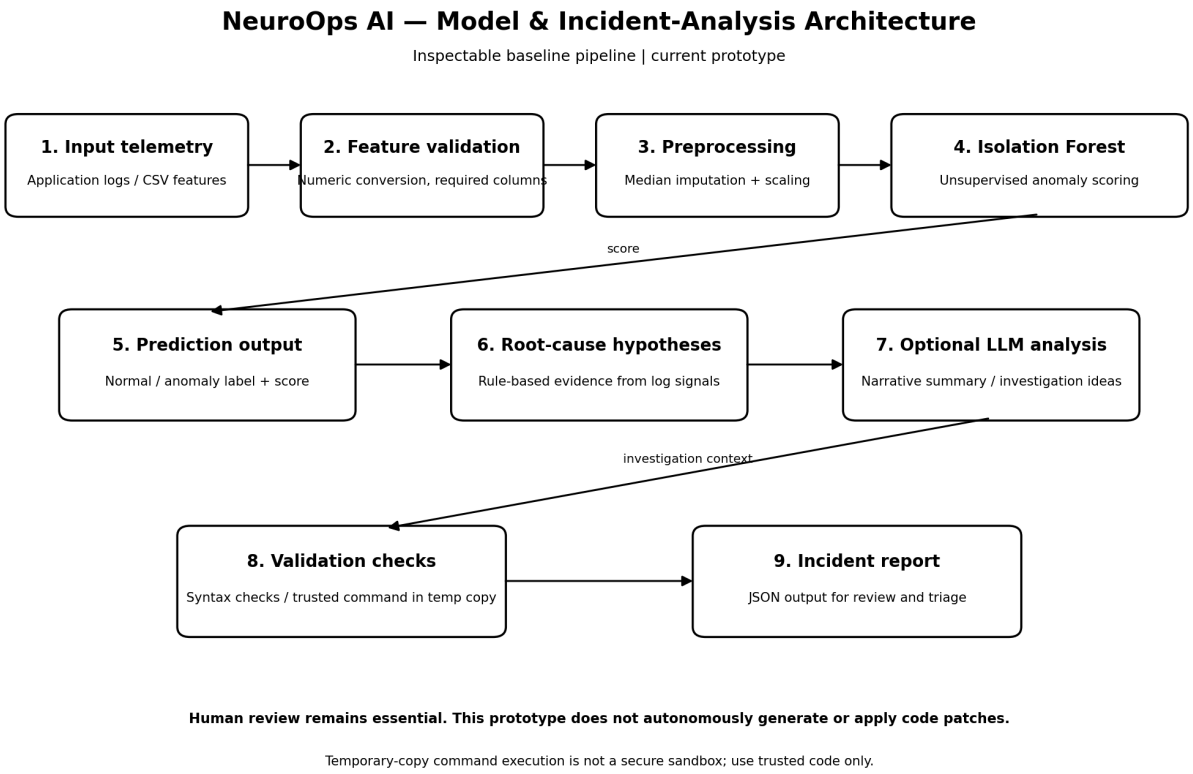
NEUROOPS AI

Model Architecture & Engineering Guide

An inspectable software-reliability prototype combining anomaly detection, root-cause hypotheses, optional LLM analysis, validation checks, and incident reporting.



Current implementation: scikit-learn Isolation Forest baseline. This document distinguishes implemented functionality from future roadmap items.

1. Executive Summary

NeuroOps AI is a prototype for investigating unusual software behaviour. It uses an unsupervised Isolation Forest model to flag numeric observations that differ from a learned baseline. Separate rule-based logic proposes possible causes from log evidence; optional LLM analysis can produce an investigation summary; repository checks and JSON reports help an engineer review the outcome.

Scope honesty: the current project is not yet a trained deep-learning system and does not autonomously generate or apply code patches. Those are potential future milestones.

2. Model Inputs

Feature	Meaning	Example
error_count	Error events in a defined observation window	3
warning_count	Warning events in the same window	8
latency_ms	Operation/request latency in milliseconds	420
response_size_bytes	Response payload size in bytes	12000

Feature definitions must be consistent across training and prediction. The examples are illustrative, not production thresholds or measured results.

3. Training and Prediction

Install project dependencies:

```
python -m pip install -r requirements.txt
```

Run the demonstration:

```
python model.py demo
```

Train on a CSV representing normal baseline behaviour:

```
python model.py train --train-csv normal_baseline.csv --model-out artifacts/anomaly_model.joblib
--contamination 0.05
```

Score a new CSV:

```
python model.py predict --model artifacts/anomaly_model.joblib --predict-csv new_observations.csv
--output predictions.csv
```

The `contamination` value is a tuning assumption about the expected outlier share. It is not an accuracy metric. The decision score is not a probability.

4. How to Evaluate It

• Use labelled incident data and a time-aware holdout set.

• Measure precision, recall, F1, false-positive rate, and detection delay.

• Break results down by service, incident category, and deployment period.

• Compare rules-only, model-only, and combined approaches.

• Monitor data drift and periodically review the baseline.

5. Security Boundaries

• Never commit API keys, credentials, private logs, or customer data.

• Review data before sending incident details to an external LLM provider.

• Treat LLM output as untrusted suggestions and require human review.

• Heuristic secret detection can miss credentials and can produce false positives.

• Temporary-copy command execution is not a secure sandbox. Never use it to execute untrusted code.

• Load serialized model artifacts only from trusted sources.

6. Current State vs. Roadmap

Implemented baseline	Potential future work
Isolation Forest training and prediction	Deep-learning or sequence-based detector
Rule-based root-cause hypotheses	Specialized investigation/remediation agents
Optional LLM-assisted summary	Human-reviewed patch generation and validation
Basic repository checks and JSON reports	CI integration and hardened sandbox

Recommended engineering milestone: create a reproducible labelled dataset, establish a baseline, publish measured evaluation results, and add tests for leakage, drift, and failure cases before making performance claims.